TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Jandry Julian Andrade Arteaga

PARALELO: A

SEMESTRE: QUINTO

ACTIVIDAD: Actividad #1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades

Actividad # 1 Estudio de Caso - Sistema de Control de Rescate y Adopción Animal.

MI ANÁLISIS

1. Análisis del escenario que hice:

Yo entendí que el sistema debe controlar desde que un ciudadano reporta un animal hasta su adopción. Identifiqué 4 actores: Ciudadano que reporta, Voluntario que apoya, Veterinario que evalúa y Administrador que asigna y cambia estados.
Requisitos que prioricé: RF04 Clasificar por especie, RF05 Crear expediente especializado y RF11 Notificar nuevos rescates. Además mantener historial y trazabilidad.

2. Modelo UML que propuse

Caso de uso: Registrar rescate, Clasificar, Crear expediente, Asignar responsable, Actualizar estado, Consultar historial.

Clases: Separé Animal de CasoRescate porque un mismo animal puede tener varios casos a lo largo del tiempo.
ExpedienteAnimal abstracto con hijos ExpedientePerro, ExpedienteGato, ExpedienteFauna.

Relaciones: Asociación Animal-CasoRescate, Composición CasoRescate-Asignación y Composición CasoRescate-HistorialEstado.

Mi código PlantText:

@Startum1
abstract class ExpedienteAnimal
class ExpedientePerro
class ExpedienteGato
ExpedienteAnimal <|-- ExpedientePerro
Animal "1" -- "0..*" CasoRescate
CasoRescate *-- Asignacion
CasoRescate *-- HistorialEstado
@enduml

3. SOLID aplicado por mí

S: Separé responsabilidades. O: ExpedienteAnimal abierto a extensión. L: Cada expediente cumple contrato. I: Interfaces pequeñas Notificador, Observador, RepositorioCasos. D: Motor depende de abstracciones no de MySQL.

4. Patrones que implementé

Singleton MotorGestionCasos:
1. Instancia única para coordinar. Constructor privado y obtenerInstancia(). Evita motores duplicados.

2. Observer RF11: ObservadorCaso y NotificadorCasos. Notifico solo después de guardar bien en BD. Puedo agregar email o móvil sin tocar registro.

3.Factory Method RF04-05:
FabricaExpediente evita ifs dispersos. Si agrego especie nueva, solo creo fábrica nueva.

4. MVC: Vista formulario, Controlador valida, Modelo CasoRescate, Repositorio interfaz y BD. Separa presentación de lógica.

5. Secuencias: Registrar caso combina los 4 patrones. Asignar responsable valida transición de estado en transacción.

6. Arquitectura y Gitflow

Capas: Presentación, Control, Dominio, Persistencia, BD.

Mi trabajo en Gitflow en github.com
Main estable, develop integración, feature/analisisrequisitos, feature/diagramasuml, feature/patronesdiseno, feature/arquitectura. Commits: feat, merge, release.

7. Conclusión personal

Separar Animal de CasoRescate da claridad. Los patrones se complementan. SOLID y capas hacen el sistema mantenible. Gitflow me dio trazabilidad real.

Evidencias

(Código de la ejecución de los ejercicios, enlace drive donde se encuentren los mismos y Capturas de pantalla claras y relevantes, donde evidencie el proceso realizado).

Bibliografía

8. BIBLIOGRAFÍAS QUE USÉ PARA MI INVESTIGACIÓN

1 Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. - Lo usé para Singleton, Observer y Factory Method.

2. Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall. - Para principios SOLID y separación por capas.

3. Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley. - Para Repositorio y MVC.

4. Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley. - Para diagramas de casos de uso, clases y secuencia.

5. Driessen, V. (2010). A successful Git branching model. Nvie.com. Recuperado de nvie.com - Para modelo Gitflow con main, develop y feature/_.

6. Sommerville, I. (2011). _Ingeniería de Software* (9na ed.). Pearson. - Para análisis de actores y requerimientos.